Большие потоки трафика и управление прерываниями в Linux

В этой заметке я опишу методы увеличения производительности линуксового маршрутизатора. Для меня эта тема стала актуальна, когда проходящий сетевой трафик через один линуксовый маршрутизатор стал достаточно высоким (>150 Мбит/с, > 50 Kpps). Маршрутизатор помимо роутинга еще занимается шейпированием и выступает в качестве файрволла.

Для высоких нагрузок стоит использовать сетевые карты Intel, на базе чипсетов 82575/82576 (Gigabit), 82598/82599 (10 Gigabit), или им подобные. Их прелесть в том, что они создают восемь очередей обработки прерываний на один интерфейс – четыре на rx и четыре на tx (возможно технологии RPS/RFS, появившиеся в ядре 2.6.35 сделают то же самое и для обычных сетевых карт). Также эти чипы неплохо ускоряют обработку трафика на аппаратном уровне.

Для начала посмотрите содержимое `/proc/interrupts`, в этом файле можно увидеть что вызывает прерывания и какие ядра занимаются их обработкой.

```
# cat /proc/interrupts
           CPU0       CPU1       CPU2       CPU3
  0:         53          1          9        336   IO-APIC-edge      timer
  1:          0          0          0          2   IO-APIC-edge      i8042
  7:          1          0          0          0   IO-APIC-edge
  8:          0          0          0         75   IO-APIC-edge      rtc0
  9:          0          0          0          0   IO-APIC-fasteoi   acpi
 12:          0          0          0          4   IO-APIC-edge      i8042
 14:          0          0          0        127   IO-APIC-edge      pata_amd
 15:          0          0          0          0   IO-APIC-edge      pata_amd
 18:        150       1497      12301     473020   IO-APIC-fasteoi   ioc0
 21:          0          0          0          0   IO-APIC-fasteoi   sata_nv
 22:          0          0         15       2613   IO-APIC-fasteoi   sata_nv,
ohci_hcd:usb2
 23:          0          0          0          2   IO-APIC-fasteoi   sata_nv,
ehci_hcd:usb1
 45:          0          0          0          1   PCI-MSI-edge      eth0
 46:  138902469      21349     251748    4223124   PCI-MSI-edge      eth0-rx-0
 47:  137306753      19896     260291    4741413   PCI-MSI-edge      eth0-rx-1
 48:       2916  137767992     248035    4559088   PCI-MSI-edge      eth0-rx-2
 49:       2860  138565213     244363    4627970   PCI-MSI-edge      eth0-rx-3
 50:       2584      14822  118410604    3576451   PCI-MSI-edge      eth0-tx-0
 51:       2175      15115  118588846    3440065   PCI-MSI-edge      eth0-tx-1
 52:       2197      14343     166912  121908883   PCI-MSI-edge      eth0-tx-2
 53:       1976      13245     157108  120248855   PCI-MSI-edge      eth0-tx-3
 54:          0          0          0          1   PCI-MSI-edge      eth1
 55:       3127      19377  122741196    3641483   PCI-MSI-edge      eth1-rx-0
 56:       2581      18447  123601063    3865515   PCI-MSI-edge      eth1-rx-1
 57:       2470      17277     183535  126715932   PCI-MSI-edge      eth1-rx-2
 58:       2543      16913     173988  126962081   PCI-MSI-edge      eth1-rx-3
 59:  128433517      11953     148762    4230122   PCI-MSI-edge      eth1-tx-0
 60:  127590592      12028     142929    4160472   PCI-MSI-edge      eth1-tx-1
 61:       1713  129757168     136431    4134936   PCI-MSI-edge      eth1-tx-2
 62:       1854  126685399     122532    3785799   PCI-MSI-edge      eth1-tx-3
NMI:          0          0          0          0   Non-maskable interrupts
```

```
LOC:  418232812  425024243  572346635  662126626   Local timer interrupts
SPU:          0          0          0          0   Spurious interrupts
PMI:          0          0          0          0   Performance monitoring
interrupts
PND:          0          0          0          0   Performance pending work
RES:   94005109   96169918   19305366    4460077   Rescheduling interrupts
CAL:         49         34         39         29   Function call interrupts
TLB:      66588     144427     131671      91212   TLB shootdowns
TRM:          0          0          0          0   Thermal event interrupts
THR:          0          0          0          0   Threshold APIC interrupts
MCE:          0          0          0          0   Machine check exceptions
MCP:        199        199        199        199   Machine check polls
ERR:          1
MIS:          0
```

В данном примере используются сетевые карты Intel 82576. Здесь видно, что сетевые прерывания распределены по ядрам равномерно. Однако, по умолчанию так не будет. Нужно раскидать прерывания по процессорам. Чтобы это сделать нужно выполнить команду `echo N > /proc/irq/X/smp_affinity`, где *N* это маска процессора (определяет какому процессору достанется прерывание), а *X* — номер прерывания, виден в первом столбце вывода */proc/interrupts*. Чтобы определить маску процессора, нужно возвести 2 в степень *cpu_N* (номер процессора) и перевести в шестнадцатиричную систему. При помощи `bc` вычисляется так: `echo "obase=16; $[2 ** $cpu_N]" | bc`. В данном примере распределение прерываний было произведено следующим образом:

```
#CPU0
echo 1 > /proc/irq/45/smp_affinity
echo 1 > /proc/irq/54/smp_affinity

echo 1 > /proc/irq/46/smp_affinity
echo 1 > /proc/irq/59/smp_affinity
echo 1 > /proc/irq/47/smp_affinity
echo 1 > /proc/irq/60/smp_affinity

#CPU1
echo 2 > /proc/irq/48/smp_affinity
echo 2 > /proc/irq/61/smp_affinity
echo 2 > /proc/irq/49/smp_affinity
echo 2 > /proc/irq/62/smp_affinity

#CPU2
echo 4 > /proc/irq/50/smp_affinity
echo 4 > /proc/irq/55/smp_affinity
echo 4 > /proc/irq/51/smp_affinity
echo 4 > /proc/irq/56/smp_affinity

#CPU3
echo 8 > /proc/irq/52/smp_affinity
echo 8 > /proc/irq/57/smp_affinity
echo 8 > /proc/irq/53/smp_affinity
echo 8 > /proc/irq/58/smp_affinity
```

Также, если маршрутизатор имеет два интерфейса, один на вход, другой на выход (классическая схема), то rx с одного интерфейса следует группировать с tx другого интерфейса на одном ядре процессора. Например, в данном случае прерывания *46 (eth0-rx-0)*

и *59 (eth1-tx-0)* были определены на одно ядро.

Еще одним весьма важным параметром является задержка между прерываниями. Посмотреть текущее значение можно при помощи `ethtool -c ethN`, параметры *rx-usecs* и *tx-usecs*. Чем больше значение, тем выше задержка, но тем меньше нагрузка на процессор.

Пробуйте уменьшать это значение в часы пик вплоть до ноля.

При подготовке в эксплуатацию сервера с Intel Xeon E5520 (8 ядер, каждое с HyperThreading) я выбрал такую схему распределения прерываний:

```
#CPU6
echo 40 > /proc/irq/71/smp_affinity
echo 40 > /proc/irq/84/smp_affinity

#CPU7
echo 80 > /proc/irq/72/smp_affinity
echo 80 > /proc/irq/85/smp_affinity

#CPU8
echo 100 > /proc/irq/73/smp_affinity
echo 100 > /proc/irq/86/smp_affinity

#CPU9
echo 200 > /proc/irq/74/smp_affinity
echo 200 > /proc/irq/87/smp_affinity

#CPU10
echo 400 > /proc/irq/75/smp_affinity
echo 400 > /proc/irq/80/smp_affinity

#CPU11
echo 800 > /proc/irq/76/smp_affinity
echo 800 > /proc/irq/81/smp_affinity

#CPU12
echo 1000 > /proc/irq/77/smp_affinity
echo 1000 > /proc/irq/82/smp_affinity

#CPU13
echo 2000 > /proc/irq/78/smp_affinity
echo 2000 > /proc/irq/83/smp_affinity

#CPU14
echo 4000 > /proc/irq/70/smp_affinity
#CPU15
echo 8000 > /proc/irq/79/smp_affinity
```

/proc/interrupts на этом сервере без нагрузки можно посмотреть тут. Не привожу это в заметке из-за громоздкости

UPD:

Если сервер работает только маршрутизатором, то тюнинг TCP стека особого значения не имеет. Однако есть параметры sysctl, которые позволяют увеличить размер кэша ARP, что может быть актуальным. При проблеме с размером ARP-кэша в dmesg будет сообщение «Neighbour table overflow».

Например:

```
net.ipv4.neigh.default.gc_thresh1 = 1024
```

```
net.ipv4.neigh.default.gc_thresh2 = 2048
net.ipv4.neigh.default.gc_thresh3 = 4096
```

Описание параметров:

gc_thresh1 — минимальное количество записей, которые должны быть в ARP-кэше. Если количество записей меньше, чем это значение, то сборщик мусора не будет очищать ARP-кэш.

gc_thresh2 — мягкое ограничение количества записей в ARP-кэше. Если количество записей достигнет этого значения, то сборщик мусора запустится в течение 5 секунд.

gc_thresh3 — жесткое ограничение количества записей в ARP-кэше. Если количество записей достигнет этого значения, то сборщик мусора незамедлительно запустится.